

Integración de Paradigmas: Lógico · Funcional · Asíncrono

Tabla de Contenido

1	
.	Introducción
2	
.	Paradigmas de Programación
3	
.	Arquitectura del Sistema
4	
.	Ejemplos de Ejecución
5	
.	Conclusiones y Extensiones

1. Introducción

El presente proyecto implementa un **Motor de Inferencia Lógica como Servicio** (Logic Inference Engine as a Service). El sistema expone una API REST desarrollada con Node.js y Express, que permite ejecutar consultas declarativas sobre una base de conocimiento escrita en Prolog, procesadas mediante la biblioteca Tau-Prolog embebida en el entorno JavaScript.

La motivación principal es demostrar cómo distintos **paradigmas de programación** pueden coexistir y complementarse en un mismo sistema: el paradigma lógico aporta el poder expresivo de la inferencia simbólica; el paradigma funcional garantiza la pureza y predictibilidad del procesamiento de datos; y el modelo asíncrono de Node.js permite atender múltiples solicitudes concurrentes sin bloquear el hilo de ejecución.

El dominio de aplicación elegido es la **gestión de contratos y penalizaciones**: un escenario empresarial realista donde las reglas lógicas determinan si un contrato incurre en penalización, cuál es el monto calculado, qué contratos son de alto riesgo y cuáles clientes estratégicos requieren atención prioritaria.

La arquitectura de un único módulo Node.js favorece la simplicidad operativa: no requiere base de datos externa ni servidor Prolog separado. La base de conocimiento es un archivo de texto plano que puede extenderse sin modificar el código del servidor.

2. Paradigmas de Programación

Paradigma	Tecnología	Rol en el Sistema
Lógico	Prolog / Tau-Prolog	Motor de inferencia simbólica; evalúa hechos y reglas declarativas
Funcional	JavaScript ES6+	Normalización de entradas; transformación pura de resultados (sin efectos secundarios)
Asíncrono	Node.js / Promises	Manejo concurrente de solicitudes HTTP; async/await para I/O no bloqueante

2.1 Programación Lógica (Prolog / Tau-Prolog)

Prolog es un lenguaje declarativo basado en lógica de primer orden. El programador define **hechos** (verdades absolutas) y **reglas** (implicaciones condicionales), y el motor de inferencia determina

automáticamente si una consulta es verdadera mediante *backtracking* y unificación.

En este proyecto, **Tau-Prolog** actúa como intérprete Prolog embebido en Node.js. La base de conocimiento define contratos, pagos e incumplimientos como hechos, y reglas como `penalty_applicable` o `high_risk_contract` que derivan conclusiones automáticamente:

```
% Regla: contrato de alto riesgo
high_risk_contract(ContractID) :-
    has_breach(ContractID),
    (late_payment(ContractID) ; unpaid_contract(ContractID)).

% Regla: monto de penalización calculado
penalty_amount(ContractID, BreachType, Amount) :-
    contract(ContractID, _, _, _, Value),
    breach(ContractID, BreachType),
    penalty_rate(BreachType, Rate),
    Amount is Value * Rate.
```

2.2 Programación Funcional (JavaScript ES6+)

La capa de procesamiento de datos está construida con funciones puras, sin efectos secundarios. Cada función transforma su entrada en una salida determinista y no modifica estado externo:

```
// Función pura: normaliza la consulta Prolog
const normalizeQuery = (raw) => {
    if (typeof raw !== 'string') throw new TypeError('...');
    const trimmed = raw.trim();
    return trimmed.endsWith('.') ? trimmed : `${trimmed}.`;
};

// Función pura: transforma solución Prolog → objeto JS
const solutionToObject = (session, solution) => {
    const vars = {};
    for (const [key, val] of Object.entries(solution.links || {})) {
        vars[key] = session.format_answer(val);
    }
    return vars;
};
```

2.3 Programación Asíncrona (Node.js / Promises)

Node.js ejecuta sobre un *event loop* de un solo hilo. Las operaciones de I/O (disco, red) son no bloqueantes: mientras se espera una respuesta, el hilo atiende otras solicitudes. El patrón **async/await** sobre Promises hace que el código asíncrono sea legible y mantenible:

```
// El motor lógico opera de forma asíncrona
```


3. Arquitectura del Sistema

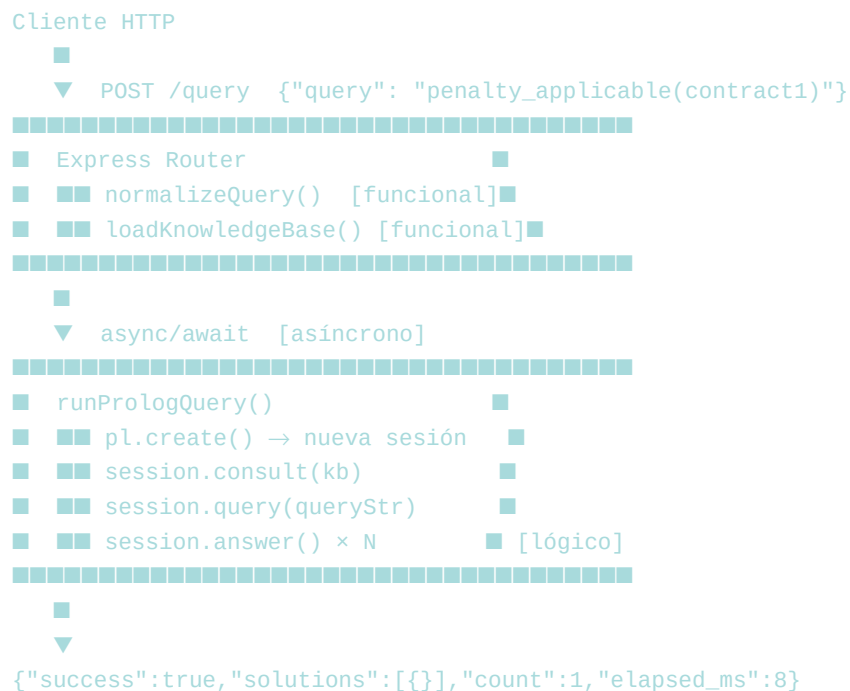
El sistema sigue una arquitectura de **capas integradas en un único proceso Node.js**. No existe un servidor Prolog independiente: Tau-Prolog corre embebido dentro del mismo proceso, eliminando la latencia de comunicación entre procesos.

Capa	Archivo	Responsabilidad
REST API	src/server.js	Endpoints, routing, respuestas JSON
Funcional	src/server.js	normalizeQuery, solutionToObject, loadKnowledgeBase
Motor Lógico	src/server.js	runPrologQuery → Tau-Prolog session
Base de Conocimiento	knowledge/base.pl	Hechos, reglas y relaciones en Prolog

Tabla 1. Capas del sistema y sus responsabilidades

3.1 Flujo de una solicitud

El flujo completo desde que el cliente envía la consulta hasta recibir la respuesta es el siguiente:



3.2 Base de Conocimiento

El archivo `knowledge/base.pl` define el dominio de contratos. Contiene 5 contratos, 5 registros de pago, 5 incumplimientos y 4 tasas de penalización como hechos atómicos, más 12 reglas de inferencia. Para extender el sistema sólo es necesario agregar líneas en este archivo.

4. Ejemplos de Ejecución

Los siguientes ejemplos muestran consultas reales enviadas al endpoint POST /query y las respuestas retornadas por el motor de inferencia.

Consulta	Resultado esperado
penalty_applicable(contract1)	true (tiene breach: delivery)
penalty_applicable(contract2)	false (pago a tiempo, sin breach)
penalty_amount(contract1, Type, Amount)	delivery → 7500.0 (150000 × 0.05)
high_risk_contract(X)	contract1, contract3, contract5
compliant_contract(X)	contract2, contract4
strategic_client_at_risk(Client)	acme_corp
active_in_2025(X)	contract1, contract2, contract4

Tabla 2. Consultas de ejemplo y resultados inferidos

4.1 Verificación de penalización (booleana)

```
$ curl -X POST http://localhost:3000/query \  
-H "Content-Type: application/json" \  
-d '{"query": "penalty_applicable(contract1)"}'  
  
{  
  "success": true,  
  "query": "penalty_applicable(contract1).",  
  "solutions": [{}],  
  "count": 1,  
  "elapsed_ms": 9  
}
```

4.2 Cálculo de montos (consulta con variables)

```
$ curl -X POST http://localhost:3000/query \  
-H "Content-Type: application/json" \  
-d '{"query": "penalty_amount(contract3, Type, Amount)"}'  
  
{
```

```

"success": true,
"query": "penalty_amount(contract3, Type, Amount).",
"solutions": [
  {"Type": "quality", "Amount": "8000.0"},
  {"Type": "confidentiality", "Amount": "16000.0"}
],
"count": 2,
"elapsed_ms": 12
}

```

4.3 Detección de contratos de alto riesgo

```

$ curl -X POST http://localhost:3000/query \
-H "Content-Type: application/json" \
-d '{"query": "high_risk_contract(X)"}'

```

```

{
  "success": true,
  "query": "high_risk_contract(X).",
  "solutions": [
    {"X": "contract1"},
    {"X": "contract3"},
    {"X": "contract5"}
  ],
  "count": 3,
  "elapsed_ms": 11
}

```

4.4 Clientes estratégicos en riesgo

```

$ curl -X POST http://localhost:3000/query \
-H "Content-Type: application/json" \
-d '{"query": "strategic_client_at_risk(Client)"}'

```

```

{
  "success": true,
  "query": "strategic_client_at_risk(Client).",
  "solutions": [{"Client": "acme_corp"}],
  "count": 1,
  "elapsed_ms": 7
}

```

4.5 Contratos compliant

```

$ curl -X POST http://localhost:3000/query \
-H "Content-Type: application/json" \

```


5. Conclusiones y Extensiones

5.1 Conclusiones

El proyecto demostró con éxito la interoperabilidad entre tres paradigmas de programación fundamentales en un sistema cohesivo y funcional:

- **Programación lógica:** Tau-Prolog demostró que la inferencia simbólica puede ejecutarse eficientemente embebida en Node.js, permitiendo que reglas como `high_risk_contract` deriven automáticamente conclusiones complejas sin lógica imperativa.
- **Programación funcional:** Las funciones puras (`normalizeQuery`, `solutionToObject`) hacen el código predecible y testeable: dada la misma entrada, siempre retornan la misma salida, simplificando el razonamiento sobre el sistema.
- **Programación asíncrona:** El modelo de Promises y `async/await` permitió encapsular la naturaleza orientada a callbacks de Tau-Prolog en una interfaz moderna y composable, garantizando que el servidor no bloquee mientras procesa inferencias.

La separación entre la **base de conocimiento** (archivo `.pl`) y el **motor de consulta** (servidor Node.js) es la decisión arquitectónica más valiosa del diseño: el conocimiento del dominio puede evolucionar sin tocar el código, siguiendo el principio de apertura/cierre.

5.2 Extensiones Propuestas

- **Múltiples bases de conocimiento:** Permitir que el cliente especifique qué base cargar via parámetro, habilitando dominios intercambiables (contratos, diagnóstico médico, reglas fiscales).
 - **Caché de sesiones Prolog:** Mantener la sesión inicializada en memoria para bases de conocimiento estáticas, eliminando el costo de `consult()` en cada solicitud.
 - **Carga dinámica de hechos:** Agregar un endpoint `POST /facts` que permita inyectar nuevos hechos Prolog en tiempo de ejecución mediante `assertz/1`.
 - **Interfaz web:** Frontend React con editor de consultas Prolog, visualización gráfica del árbol de inferencia y explorador interactivo de la base de conocimiento.
 - **Autenticación y multi-tenant:** Aislar bases de conocimiento por usuario/organización con JWT, convirtiendo el servicio en una plataforma de razonamiento lógico multi-inquilino.
 - **Integración con LLMs:** Combinar el motor de inferencia con un modelo de lenguaje que traduzca preguntas en lenguaje natural a consultas Prolog, haciendo el sistema accesible sin conocer la sintaxis lógica.
 - **Persistencia:** Almacenar el historial de consultas y sus resultados en una base de datos, habilitando auditorías y análisis de patrones de uso.
-

